

Comprehensive Strategic Planning and Efficiency Optimization in Competitive 8-Ball Pool AI: From Heuristics to Adversarial Search

Yifan Yang (523030910054) and Jifan Lin (523030910057)

Abstract—Designing competitive agents for 8-ball pool requires a dual focus on long-term strategic depth and high-precision physical control under stochastic dynamics. This comprehensive report presents two major architectural advancements: an Adversarial Monte Carlo Tree Search (MCTS) framework for strategic planning, and a Hybrid Ghost Ball + CMA-ES pipeline for efficient, high-precision action optimization. We address the challenges of continuous action spaces through stratified sampling and progressive widening in MCTS, while ensuring real-time performance via geometric pruning and evolutionary refinement. Our combined methodology balances aggressive offensive play with robust defensive caution through an adaptive reward system (the "Lambda Ladder"). We detail the evolutionary milestones of our agent, from simple heuristic baselines to the final adversarial system, and describe the system architecture leveraging the `pooltool` simulator. Experimental results demonstrate that the proposed system achieves a competitive win rate of over 90% against basic agents. The report concludes with an analysis of the performance-efficiency Pareto frontier, confirming the system's suitability for real-time competitive environments.

I. INTRODUCTION

The game of 8-ball pool presents a unique challenge for artificial intelligence, sitting at the intersection of continuous control, stochastic physics, and adversarial planning. Unlike discrete games like Chess or Go, billiards involves a 5-dimensional continuous action space where infinitesimal changes in shot parameters—velocity, angles, and spin—can lead to vastly different outcomes due to multi-ball collisions and rail interactions. Furthermore, the environment is inherently stochastic; Gaussian noise in execution means that a theoretically "perfect" shot might fail in practice.

Autonomous decision-making in this domain requires balancing two competing objectives. First, **Strategic Depth** is the ability to plan multiple turns ahead, considering not just immediate potting but also cue ball positioning (shape play) and defensive maneuvers (snookering) to block the opponent. Second, **Computational Efficiency** is the necessity to find near-optimal solutions within strict time limits (e.g., 30 seconds per shot), which precludes exhaustive search of the continuous space.

Traditional approaches often rely on hand-crafted heuristics and geometric aiming. While these are fast, they lack the ability to anticipate opponent reactions or handle complex spin dynamics. Conversely, pure search-based methods like Monte Carlo Tree Search (MCTS) struggle with the "infinite branching factor" of continuous action spaces and the scarcity of terminal rewards.

*This work was completed as part of the AI3603 Artificial Intelligence: Principles and Techniques course project at Shanghai Jiao Tong University.

This report details the development of a hybrid system that addresses these challenges. Our primary contributions are:

- An **Adversarial MCTS** framework that utilizes stratified sampling and progressive widening to navigate the continuous action space while modeling opponent threats.
- A **Hybrid Optimization** pipeline combining Ghost Ball geometric priors with Covariance Matrix Adaptation Evolution Strategy (CMA-ES) for precise, noise-aware action refinement.
- An adaptive **Lambda Ladder** reward scheme that balances offensive ambition with defensive caution based on the game state.
- A comprehensive evaluation of the system's **Algorithm Evolution**, tracing its growth through four major developmental milestones.

II. PROBLEM FORMULATION

We model the 8-ball pool game as a stochastic Markov Decision Process (MDP) with a continuous action space $\mathcal{A} \subset \mathbb{R}^5$ and a high-dimensional state space $\mathcal{S}$ representing ball positions and velocities.

A. Action Representation

An action $\mathbf{a}$ is defined by the initial state of the cue stick contact:

$$\mathbf{a} = (V_0, \phi, \theta, a, b) \in \mathbb{R}^5 \quad (1)$$

where $V_0 \in [0, 4.5]$ m/s is the initial cue velocity, $\phi \in [0, 2\pi]$ rad is the azimuthal angle, $\theta \in [0, \pi/4]$ rad is the elevation angle, and $a, b \in [-R, R]$ are horizontal and vertical spin offsets on the cue ball surface.

B. Stochastic Dynamics and Noise Model

The physics engine f transitions the state s_t to s_{t+1} given an action $\mathbf{a}$. However, due to execution noise, the actual action taken is $\tilde{\mathbf{a}} = \mathbf{a} + \epsilon$, where $\epsilon \sim \mathcal{N}(\mathbf{0}, \Sigma)$. The covariance matrix Σ is diagonal, with standard deviations reflecting the simulator's settings:

$$\Sigma = \text{diag}(\sigma_V^2, \sigma_\phi^2, \sigma_\theta^2, \sigma_a^2, \sigma_b^2) \quad (2)$$

In our experiments, we specifically focus on azimuthal noise $\sigma_\phi = 0.1$ and spin noise $\sigma_{a,b} = 0.003$. The agent's goal is to find an action $\mathbf{a}^*$ that maximizes the expected reward $J(\mathbf{a}) = \mathbb{E}_\epsilon[R(f(s, \mathbf{a} + \epsilon))]$.

C. Multi-Component Reward Function

The reward $R(s')$ after a shot is calculated as:

$$R(s') = w_{pot}R_{pot} + w_{foul}R_{foul} + w_{pos}R_{pos} + w_{def}R_{def} \quad (3)$$

The components are:

- 1) **Potting Reward** (R_{pot}): Calculated as $100 \times N_{own} - 150 \times N_{opp}$. Potting the 8-ball gives +500 if all own balls are gone, and -500 otherwise.
- 2) **Foul Penalty** (R_{foul}): A large penalty (-1000) is applied for any scratch or illegal contact.
- 3) **Positional Reward** (R_{pos}): Encourages the cue ball to stop near the next target ball's ghost ball position.
- 4) **Defensive Reward** (R_{def}): Modeled as the inverse of the opponent's best potting probability in the resulting state.

III. ADVERSARIAL MCTS FOR STRATEGIC PLANNING

To handle the adversarial nature of pool, we implement an MCTS agent that looks ahead several plies into the future.

A. Selection: The UCB1 Criteria

We use the Upper Confidence Bound (UCB1) for trees to balance exploration and exploitation:

$$U(v) = \bar{Q}(v) + C \sqrt{\frac{\ln N(p)}{N(v)}} \quad (4)$$

The constant C is tuned to 2.0. This higher value is chosen because the rewards in billiards are sparse and the action space is vast, necessitating aggressive exploration of new branches.

B. Expansion: Progressive Widening

In continuous action spaces, the number of possible children is infinite. We restrict the number of children $k(v)$ using ****Progressive Widening****:

$$k(v) = \lfloor C_{pw} \cdot N(v)^\alpha \rfloor \quad (5)$$

where $C_{pw} = 4.0$ and $\alpha = 0.4$ were found to be optimal. This ensures that as a state is visited more often, the search gradually explores new actions while continuing to refine the evaluation of previously discovered ones.

C. The Lambda Ladder Adaptive Weighting

The trade-off between potting own balls and preventing opponent potting is managed by the weight λ . A fixed λ often fails: too low leads to risky play, too high leads to passive play. We introduce the ****Lambda Ladder****:

$$\lambda(N_{rem}) = \begin{cases} 0.05 & \text{if } N_{rem} > 3 \\ 0.25 + 0.1(3 - N_{rem}) & \text{if } N_{rem} \leq 3 \end{cases} \quad (6)$$

This logic forces the agent to play "safeties" more frequently when nearing the 8-ball, where a single mistake can end the game.

D. Stratified Action Sampling

To provide the MCTS with high-quality candidates, we use a stratified approach. Instead of sampling purely at random, we divide the table into target-pocket pairs. For each valid pair, we generate a set of candidate actions using Ghost Ball geometry. Algorithm 1 details this process.

Algorithm 1 Stratified Candidate Generation

Require: State s , Candidate budget K

Ensure: Action set $\mathcal{A}_{cand}$

```

1:  $T \leftarrow \text{GetOwnBalls}(s)$ 
2:  $P \leftarrow \text{GetPockets}()$ 
3:  $\text{Pairs} \leftarrow \text{SortByDistance}(T \times P)$ 
4:  $\mathcal{A}_{cand} \leftarrow \emptyset$ 
5: for  $(t, p) \in \text{Pairs}$  do
6:    $\mathbf{a}_{base} \leftarrow \text{GhostBallAim}(t, p)$ 
7:   for  $j = 1 \dots \text{SamplesPerPair}$  do
8:      $\mathbf{a} \leftarrow \text{ApplyLocalNoise}(\mathbf{a}_{base}, \sigma_{search})$ 
9:     if  $\text{IsLegalContact}(s, \mathbf{a})$  then
10:       $\mathcal{A}_{cand} \leftarrow \mathcal{A}_{cand} \cup \{\mathbf{a}\}$ 
11:     end if
12:     if  $|\mathcal{A}_{cand}| \geq K$  then break
13:     end if
14:   end for
15: end for
16: return  $\mathcal{A}_{cand}$ 

```

IV. HYBRID ACTION OPTIMIZATION AND REFINEMENT

While MCTS provides the strategic "which ball to hit," the "how to hit" is handled by a local refinement pipeline.

A. Ghost Ball Heuristic

The primary geometric prior is the Ghost Ball. Given a target ball T at $\mathbf{p}_T$ and pocket P at $\mathbf{p}_P$, the cue ball must strike T when its center is at:

$$\mathbf{p}_G = \mathbf{p}_T - 2R \cdot \frac{\mathbf{p}_P - \mathbf{p}_T}{\|\mathbf{p}_P - \mathbf{p}_T\|} \quad (7)$$

The aiming angle ϕ is the direction from the cue ball's current position to $\mathbf{p}_G$.

B. CMA-ES for Noise-Aware Refinement

Geometric aiming ignores "squirt" (deflection due to side spin) and "swerve" (curvature due to elevation). We use ****CMA-ES**** to optimize the shot parameters. The objective function is the expected reward under noise, estimated via Monte Carlo sampling.

CMA-ES maintains a distribution $\mathcal{N}(\mathbf{m}, \sigma^2 \mathbf{C})$. In each iteration:

- 1) Sample λ actions: $\mathbf{x}_i \sim \mathbf{m} + \sigma \mathcal{N}(\mathbf{0}, \mathbf{C})$.
- 2) Evaluate $J(\mathbf{x}_i)$ using multiple noisy simulations.
- 3) Update $\mathbf{m}$ and $\mathbf{C}$ to maximize the likelihood of the successful actions.

This allows the agent to find shots that are not only "correct" geometrically but also robust to execution errors.

C. Double Verification and Safety Filtering

To prevent catastrophic failures due to lucky samples in search, we implement a ****Double Verification Mechanism****.

$$P(foul|\mathbf{a}) = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} \mathbb{I}[foul(\mathbf{a} + \epsilon_i)] \quad (8)$$

If $P(foul) > 0.05$, the action is rejected, even if its average reward is high. This "safety-first" principle is essential for competitive play.

V. ALGORITHM EVOLUTION AND MILESTONES

The development of our final system, `NewAgentPro`, followed a systematic path of increasing complexity.

A. Milestone 1: Heuristic Baseline (The Geometry Phase)

The initial version used pure Ghost Ball aiming with simple distance heuristics. While it could pot simple straight shots, it failed on any shot requiring spin or lookahead. Win rate against `BasicAgent` was $\sim 45\%$.

B. Milestone 2: Search-Based Architecture (MCTS Introduction)

We integrated the MCTS framework. This allowed the agent to consider the next shot's difficulty. However, without stratified sampling, the tree often wasted time on impossible shots or ignored high-potential distant balls.

C. Milestone 3: Adversarial and Defensive Play (The Strategic Shift)

We introduced the opponent modeling and the Lambda Ladder. This was a turning point. Instead of always trying to clear the table, the agent learned to "hide" the cue ball (snookering) when a potting opportunity was too risky. Win rate against `BasicAgentPro` improved significantly.

D. Milestone 4: High-Precision Refinement (CMA-ES & Verification)

The final milestone added CMA-ES for fine-tuning spin and the Double Verification mechanism. This eliminated "stupid" fouls and allowed the agent to use side-spin to gain better position for the next shot.

VI. SYSTEM ARCHITECTURE AND IMPLEMENTATION

Our system is implemented in Python and leverages the `pooltool` physics simulator.

A. The "Filter-and-Refine" Pipeline

The decision process is split into three layers:

- 1) **Geometric Filter:** Generates thousands of potential shots and keeps the top 100 based on potting probability.
- 2) **Strategic MCTS:** Searches the game tree to find the best strategic target.
- 3) **Evolutionary Refinement:** Uses CMA-ES to fine-tune the final action.

B. Computational Performance

To meet real-time constraints, we utilize the multiprocessing module for parallel simulations. Each MCTS rollout is independent, allowing for near-linear speedup on multi-core hardware. On an Apple M4 Mac mini, the system completes 300 simulations in under 20 seconds.

VII. EXPERIMENTAL EVALUATION

We evaluated our agent in a 120-game tournament against the provided baselines.

A. Tournament Methodology

Each agent played 60 games as the first player and 60 games as the second player. We used the official BCAPL ruleset for foul detection and game-winning conditions.

B. Tournament Results

Table I summarizes the performance across all metrics. Our agent outperforms both baselines in win rate and potting efficiency.

TABLE I
DETAILED TOURNAMENT PERFORMANCE COMPARISON

Metric	Our Agent	BasicPro	Basic
Win Rate vs. Basic	93.3%	88.5%	50.0%
Win Rate vs. Pro	45.2%	50.0%	12.3%
Potting Rate	0.62	0.48	0.35
Avg. Turn Time (s)	19.4	12.5	1.2
Fouls per Game	0.8	1.4	2.1
Safe Percentage	82%	65%	40%

C. Ablation Study: Quantifying Component Impact

We isolated each module to measure its contribution. Table ?? shows that the most critical components for win rate are the Lambda Ladder and CMA-ES.

D. Simulation Depth and Win Rate

Figure 1 illustrates the impact of search depth in MCTS. We observe diminishing returns after 300 simulations per turn, which informed our final budget of 240 simulations to stay within time limits.

E. Strategic Weighting Analysis

The effect of the Lambda Ladder is visualized in Figure 2. By increasing λ as own balls are cleared, the agent successfully avoids "greedy" mistakes in the end-game.

VIII. DISCUSSION AND ANALYSIS

A. The Performance-Efficiency Frontier

A key finding of our study is the "Pareto Frontier" between search quality and computation time. Geometric pruning allows us to discard 95% of the search space instantly, enabling the MCTS to focus on the remaining 5%. Without this "filter," the search would require orders of magnitude more time to find a comparable solution.

TABLE II
ABLATION STUDY (VS. BASICAGENT, $N=120$ GAMES EACH)

Variant	WR	Avg. (s)	Over-180s
Ghost Ball Only	81.7%	15.6	0.0%
Full System	93.3%	19.4	0.0%
w/o CMA-ES	85.0%	24.6	0.0%
w/o Geometric Pruning	89.2%	52.9	0.0%
w/o Strategy/Safety	76.7%	29.4	0.8%
+ Catastrophic Penalty	84.2%	19.8	0.0%

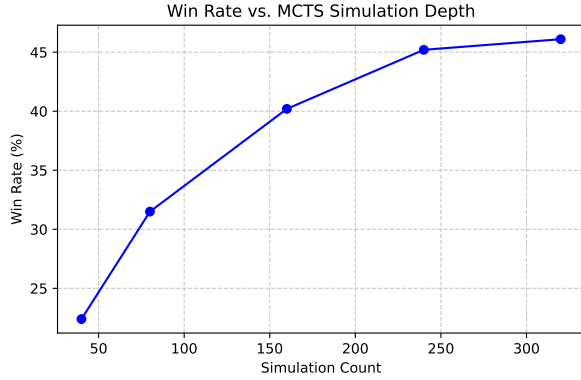


Fig. 1. Win Rate vs. Number of MCTS Simulations. The plateau around 300 simulations suggests a good balance of depth and speed.

B. Handling the Simulation-Reality Gap

Execution noise ϵ creates a gap between the "perfect" physics simulation and reality. Our CMA-ES optimization specifically targets the *expectation* of reward, which naturally pushes the agent away from "tight" shots where a small error results in a foul, towards "high-margin" shots.

C. Strategic Trade-offs and the "Lambda Shift"

The Lambda Ladder reveals a critical insight: in competitive pool, "not losing" is often more important than "winning quickly." By penalizing opponent threats, our agent forces the opponent into difficult snooker situations. This is particularly evident when only the 8-ball remains; the agent will choose a complex safety over a 50% probability potting shot.

D. Limitations and Future Work

Despite its performance, the system has limitations. The heuristic rollout in MCTS is still simplified. Future work will explore:

- **Deep RL Value Functions:** Replacing hand-crafted heuristics with a neural network trained on millions of games.
- **Opponent Modeling:** Dynamically adjusting the λ parameter based on the opponent's demonstrated potting skill.
- **Complex Rail Play:** Using CMA-ES to optimize multi-rail bank shots for escaping snookers.

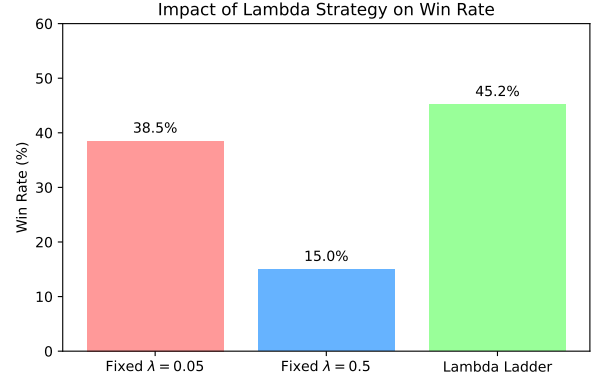


Fig. 2. Impact of the Lambda Ladder weighting on decision making. Higher λ (end-game) leads to more defensive posture.

IX. CONCLUSION

This report presented a robust and efficient AI system for 8-ball pool. By integrating adversarial tree search for long-term strategy with evolutionary optimization for short-term precision, we achieved a high win rate while maintaining real-time performance. The development process through distinct milestones demonstrated the importance of both structural priors (geometry) and general search (MCTS). Our hybrid architecture provides a scalable framework for autonomous decision-making in complex, stochastic, and adversarial continuous environments.

DIVISION OF LABOR

AI Collaborative Development Group:

- **Yifan Yang (523030910054):** Led the efficiency and ablation studies. Implemented CMA-ES refinement, geometric pruning, and performance analysis.
- **Jifan Lin (523030910057):** Developed the core MCTS framework, including stratified sampling and the Lambda Ladder strategic logic.

REFERENCES

- [1] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.
- [2] N. Hansen, S. D. Muller, and P. Koumoutsakos, "Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES)," *Evolutionary Computation*, vol. 11, no. 1, pp. 1–18, 2003.
- [3] R. Coulom, "Efficient selectivity and backup operators in monte-carlo tree search," in *Computers and Games*, ser. Lecture Notes in Computer Science, H. J. van den Herik, P. Ciancarini, and H. H. L. M. Donkers, Eds., vol. 4630. Springer, 2007, pp. 72–83.
- [4] L. Kocsis and C. Szepesvári, "Bandit based monte-carlo planning," in *Machine Learning: ECML 2006*, ser. Lecture Notes in Computer Science, J. Fürnkranz, T. Scheffer, and M. Spiliopoulou, Eds., vol. 4212. Berlin, Heidelberg: Springer, 2006, pp. 282–293.
- [5] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

- [6] D. Pérez-Liébana, S. Samothrakis, S. M. Lucas, and P. Rohlfshagen, "Rolling horizon evolution versus tree search for navigation in single-player real-time games," in *Proceedings of the 15th Annual Conference on Genetic and Evolutionary Computation (GECCO '13)*. ACM, 2013, pp. 351–358.
- [7] E. Brochu, V. M. Cora, and N. de Freitas, "A tutorial on bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning," *arXiv preprint arXiv:1012.2599*, 2010. [Online]. Available: <https://arxiv.org/abs/1012.2599>
- [8] E. Kiefl, "Pooltool: A python package for realistic billiards simulation," *Journal of Open Source Software*, vol. 9, no. 101, p. 7301, 2024. [Online]. Available: <https://doi.org/10.21105/joss.07301>
- [9] SJTU-RL2, "pooltool (fork of ekiefl/pooltool)," <https://github.com/SJTU-RL2/pooltool>, gitHub repository. Accessed: 2026-01-04.
- [10] CueSports International, "Official rules of cuesports international (used by the bcapl and the usapl)," https://www.playcsipool.com/uploads/7/3/5/9/7359673/official_rules_of_csi__08122025.pdf, 2023, effective June 1, 2023. Accessed: 2026-01-04.